

Spherical Easel Tours



Wyatt Watson
Sam Rios
Ben Turner
Jason Bos

Sponsors:
Dr. Dickinson
Dr. Dulimarta

Table of Contents

Overview	...3
Languages	...3
Libraries, SDKs, and APIs	...4
Code Repository Organization	...5
System Organization	...6
Development	...6
Non-trivial Requirement	...7
Mockups	..7
Work Policies	...8
Testing	...8
Deliverance and Milestones	...9

Overview

Online tools for in the classroom learning for math have come a long way. One of the most notable platforms that includes such tools is Desmos, which provides graphing and scientific calculators. These tools come with online tutorials that help guide new users on how to access specific features within their platform. Spherical Easel takes the concept of spherical geometry and puts it to the test. This tool allows students/researchers to facilitate the learning (and teaching) of spherical geometry. With a tool as vast as Spherical Easel, there are many different options and features that allow for many different use cases. This can be very daunting for new users of the website. Our main focus on this project is to be able to create interactive tours/tutorials that will help guide users on how to use the platform. Our goal is to make it as intuitive as possible, making sure that the user can get comfortable using the web app.

The sponsors described the project as fairly open-ended, with the hope that the structure created during this project can be built on later for a larger UI tour and help system. Because of this, one of the main challenges will be creating a system that can be reused for many different tours instead of creating each tour separately. Our approach to this challenge will be an iterative one as we learn to work with the tools and code base while making simple tours. The more we understand the website's systems and document them, the clearer it will be how to lay the groundwork for a future UI tour and help system. The prospectus identifies a streamlined tour experience, tour popups, a reusable framework, documentation, internationalization, and possible recording and Firebase storage as the primary areas of the project.

Languages

Spherical Easel is an existing web application written primarily in TypeScript and Vue. Because our project is being added directly into the existing application, we will continue using TypeScript rather than introducing a new programming language.

The team already has experience with several of the technologies required for the project. Specifically, experience with Vue, TypeScript, and JavaScript. The project will allow the team members with previous Vue and TypeScript experience to continue building those skills while the other team members gain more experience with them. The prospectus also identifies working with a larger existing codebase, TypeScript, and UI/UX development as technical growth areas for members of the team.

The project will also make use of HTML and CSS through Vue single-file components. These are already part of the Spherical Easel application and will be used when creating and styling the tour interface.

Libraries, SDKs, and APIs

VueJS

VueJS is the existing framework used to develop the Spherical Easel web application. The tour system will be implemented using Vue components so it can interact directly with the existing application.

Vuetify

Vuetify is the UI framework already used by Spherical Easel. We will use Vuetify for the tour popups and other interface elements. The prospectus specifically identifies Vuetify as the framework that will be used for tooltips and tour popups.

Vue Router

Vue Router is used by the existing application to control different views. Our initial tour prototype has been added as a route so that the tour system can be tested directly inside Spherical Easel.

Pinia

Pinia is used by Spherical Easel for application state management. The tour system will potentially use existing application state to determine when a user has completed a required action.

Vue I18n

Spherical Easel already uses Vue I18n for translations. Tour messages will use the existing translation system instead of adding a separate translation system. This will allow the tours to support the same languages as the rest of the application.

Firebase

Google Firebase is already used by Spherical Easel for user accounts and cloud functionality. Firebase may also be used to store and share created tours. This part of the project is considered a possible feature rather than a required part of the initial tour framework.

Vite

Vite is used to build and serve the existing application during development. The current repository uses Vite together with the Vue and Vuetify plugins.

Vitest

Vitest is already included in the repository for testing. The project also includes Vue Test Utils and a configured DOM testing environment. This gives us an existing testing framework that can be used when testing the tour components and framework.

Code Repository Organization

Our project will use GitHub to manage the source code. The Spherical Easel project is already maintained in a Git repository, and our project is being developed in a fork of that repository.

Main Branch

The main branch will contain the finished and functional version of the project. Code will only be merged into main after it has been integrated, reviewed, and tested by the team. This branch will represent the most stable version of the project.

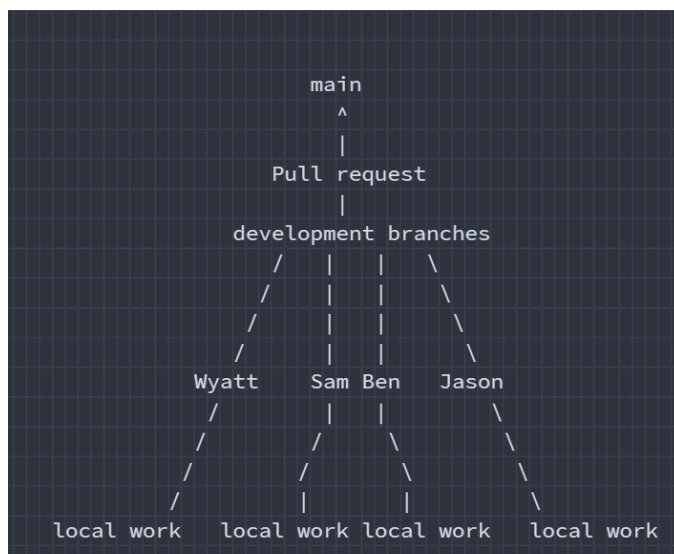
Development Branch

The development branch will be used to integrate and test active development work. This will be the branch used to combine work from individual team members before it is moved into main. Features can be demonstrated and tested by the entire team from this branch without putting unfinished work into the stable branch.

Personal Feature Branches

Each team member will create their own feature branch from development when working on a task. These branches will be used for local development and will be associated with a specific Linear issue. An example branch name would be: Wyatt/CIS467-23-point-tour

Once a feature is ready to be tested by the team, the developer will push the branch to GitHub and create a pull request into development. Another team member will review the changes before they are merged. This allows the team to test new functionality together while continuing to keep main stable.



This workflow will allow us to integrate work regularly while still giving individual team members a separate place to develop their features. It also gives the team a shared branch where new work can be demonstrated and tested before becoming part of the stable version of the project.

Linear will be used to manage the project backlog, assign tasks, and track progress throughout each sprint. GitHub will be used for source code, branches, pull requests, code reviews, and version history.

The team will merge work frequently rather than waiting until the end of a sprint. This will allow us to identify integration problems earlier and keep the project in a working state throughout development.

System Organization

The tour system will be integrated into the existing Spherical Easel Vue application. This means that the tour system will not require a separate client application or server for its core functionality.

The system will consist of a reusable tour framework and individual tour definitions. A tour definition will describe the steps of the tour, including the message, target interface element, and expected user action. The tour framework will handle the current tour, current step, popup display, UI highlighting, navigation, and completion conditions.

The most important design decision is that the tour framework will remain separate from the individual tour definitions. This means that adding a new tour should primarily involve defining its steps rather than creating a new implementation of the complete tour system.

This approach was selected because the sponsors want the project to provide a structure that can be expanded in the future. A framework that separates tour data from the functionality that displays and progresses through tours will make it easier to add new tutorials and change existing ones.

The current repository already contains the major infrastructure required for this approach. The Vue application, state management, Firebase configuration, Vite development system, and testing tools are already part of the existing codebase.

Development

The tour system will be developed in local environments using the existing Spherical Easel development setup. Bun will be used to install the project dependencies, and Vite will be used to run the application locally.

The repository already contains the configuration needed for development, testing, and building the application. A shared dependency configuration and lockfile will be used so that each team member can keep their environment as similar as possible.

The project also contains Docker and Netlify deployment configuration. These existing deployment options can be used when creating a stable build for demonstrations and the final release.

Non-trivial Requirement

The main non-trivial requirement of this project is creating a tour system that can respond to user actions instead of simply displaying a sequence of instructions.

A basic tooltip can display a message next to an interface element, but the tour system needs to determine whether the user actually completed the action being described before moving on to the next step.

For example, the Point Tour requires the user to activate the Point tool and click on the main sphere canvas to create a point. The system needs to recognize that the point was actually created before continuing to the next step. The Segment Tour creates another challenge because the user can create the segment in more than one way. The prospectus identifies both selecting two locations and clicking and dragging as possible methods.

Other tours require specific interactions such as holding the Shift key while creating a point or holding the Control key while creating a segment. The system therefore needs to be able to recognize different types of user actions and determine when each step has been completed.

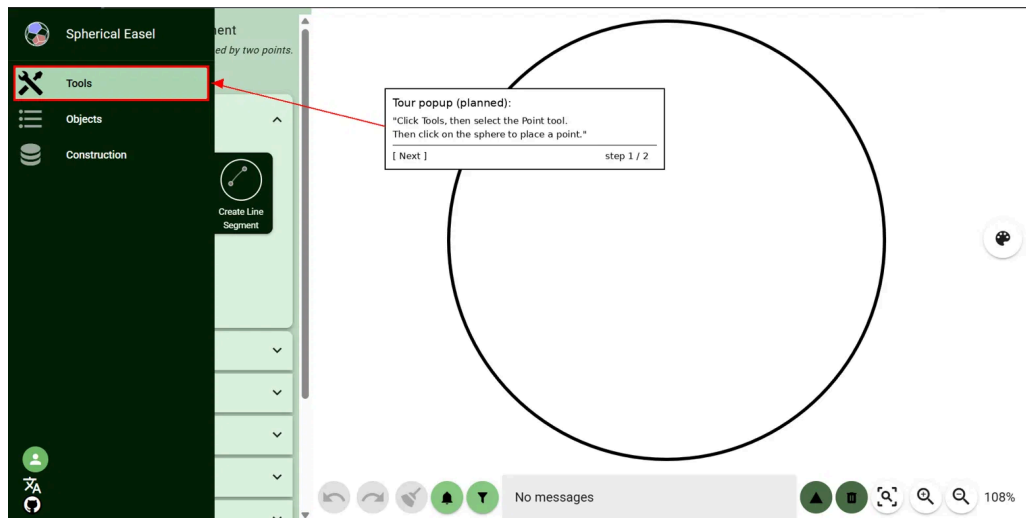
Our proposed solution is to give each tour step a completion condition. The condition will describe what needs to happen within the existing Spherical Easel application before the tour can progress.

The exact implementation of the completion conditions will be determined through prototyping. The first three sprints specifically focus on experimentation, framework development, and user action detection before the main system is finalized.

Another non-trivial requirement is making the system reusable. Instead of creating a separate implementation for every tutorial, the framework should allow new tours to reuse the same components and logic.

Mockups

Since no tour UI has been built yet, the figure below shows a planned tour popup added directly onto a screenshot of the actual Spherical Easel site. The red box highlights the Tools menu the user needs to open, and the popup shows the planned instructions message, a Next button, and a step indicator. These are positioned over the sphere where the popup would actually appear. The first tour (Point Tour) directs the user to open Tools, select the Point tool, then click the sphere canvas to place a point.



Later tours will reuse the same popup structure we will decide to use for the first tour, only changing the target element, message text, and completion condition. While the provided mockup may not be the final version of what the popups look like, they will stay consistent with the same approach described above.

Work Policies

- Sponsor contact and project management:
 - Wyatt Watson
- Presentation and Documentation deliverables:
 - Wyatt Watson
 - Sam Rios
- Development and Testing:
 - Wyatt Watson
 - Sam Rios
 - Ben Turner
 - Jason Bos

Testing

Testing will be performed throughout development instead of being left until the final sprint.

The existing repository already includes Vitest, Vue Test Utils, and a configured test environment. These tools will be used to test the tour framework and reusable components.

Unit testing will be used to test individual pieces of the tour system. This includes changing steps, checking completion conditions, loading tour definitions, and handling tour navigation.

Integration testing will be used to verify that the tour system works with the existing Spherical Easel application. This includes checking that the framework can locate the correct UI elements and respond correctly when the user performs an action.

The team will also conduct user testing. During Sprint 4, users who are unfamiliar with Spherical Easel will be asked to complete tours. Their feedback will be used to identify confusing instructions, unclear UI behavior, and usability issues.

Pull requests will also be part of the testing process. Before changes are merged, another team member will review the implementation and verify that it functions as expected.

Deliverance and Milestones

The project will be developed across five two-week sprints. Each week will have a specific deliverable or milestone that can be demonstrated to the sponsors during the weekly Tuesday meeting. The first three sprints concentrate on exploration, prototyping, and development of the reusable tour system. Sprint 4 focuses on testing and refinement, while Sprint 5 focuses on final testing, documentation, the final report, demo video, poster, and presentation.

Sprint 1

- Week 1: Development environment setup, codebase exploration, GitHub/Linear workflow
- Week 2: Tour prototypes and comparison of implementation approaches

Sprint 2

- Week 1: Tour structure, data model, and framework design
- Week 2: Basic reusable tour framework and first working tour

Sprint 3

- Week 1: User action detection, completion conditions, and dynamic progression
- Week 2: Reusable dynamic tour system and additional tours

Sprint 4

- Week 1: User testing, UX refinement, and bug identification
- Week 2: Core feature completion, testing, documentation, and stable demo build

Sprint 5

- Week 1: Final testing, documentation, architecture diagrams, and final report draft
- Week 2: Final report, demo video, poster, presentation, and final client review